SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence	**COURSE CODE: CSA17**
COURSE OUTCOME COVERED CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) Assessment Tool Weightage: 35%	

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I V. Vishwa Tej certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Vishwa Tej

1. Doctor-to-Shift Assignment Using Backtracking Search

A hospital must assign doctors D1, D2 and D3 to Morning, Afternoon and Night shifts. Each doctor must receive exactly one shift, each shift must have exactly one doctor, D1 cannot work Night, D2 must work before D3, and D3 cannot work Morning.

Constraint satisfaction formulation

Component	Definition
Variables	D1, D2, D3
Domains	Each variable has domain {Morning, Afternoon, Night}
All-different constraint	D1, D2 and D3 must be assigned different shifts
Unary constraints	D1 ≠ Night; D3 ≠ Morning
Ordering constraint	D2 must be assigned an earlier shift than D3: Morning < Afternoon < Night
Goal	Find a complete assignment satisfying every constraint

Backtracking search with constraint checks

Step	Assignment considered	Constraint check	Decision
1	D2 = Morning	Allowed; earliest shift	Accept
2	D3 = Afternoon	D2 is before D3; D3 is not Morning	Accept
3	D1 = Night	Violates D1 ≠ Night	Reject and backtrack
4	D1 = Afternoon	Conflicts with D3 = Afternoon (one doctor per shift)	Reject
5	Backtrack D3; try D3 = Night	D2 = Morning is before D3 = Night; D3 not Morning	Accept
6	D1 = Afternoon	Different from D2 and D3; D1 not Night	Accept; complete solution

Final valid schedule: Morning — D2 | Afternoon — D1 | Night — D3

All three doctors receive exactly one shift and each shift has one doctor. D1 is not assigned to Night, D3 is not assigned to Morning, and D2 works before D3. Therefore, the final schedule satisfies all stated constraints.

2. Robot Navigation in a 5 × 5 Grid Using BFS

The question specifies four-direction movement, unit cost per move, blocked obstacles, and Manhattan distance as a heuristic. However, the supplied question paper does not show the locations of the obstacles or the exact cells for S and G. To demonstrate the requested method, the following is an explicitly illustrative grid; the path would change if the intended obstacle layout differs.

Illustrative grid (assumption for demonstration)

S	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	■	■	(1,3)	(1,4)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)
(3,0)	(3,1)	■	■	(3,4)
(4,0)	(4,1)	(4,2)	(4,3)	**G**

Legend: S = start (0,0); G = goal (4,4); ■ = blocked cell. Coordinates are (row, column). Neighbours are considered in the order Right, Down, Left, Up for a reproducible trace.

Search properties

Item	Explanation
State	Robot's current grid cell (row, column)
Actions	Up, Down, Left, Right, if the destination cell is inside the grid and not blocked
Step cost	1 for every movement
BFS frontier	FIFO queue; nodes are expanded in order of discovery
Manhattan distance	$h(n) = \|row - goal_row\| + \|column - goal_column\|$; included as requested, but BFS itself does not prioritize by h(n)
Optimality	BFS finds a shortest path when every move has equal cost

2. Robot Navigation — Node Expansion and Result

Using the illustrative grid on the previous page, breadth-first search (BFS) stores discovered cells in a FIFO queue and marks them visited when enqueued. The heuristic is calculated for reference; unlike Greedy Best-First Search or A*, it does not change BFS's expansion order.

Expansion	Node removed	New valid neighbours (Right, Down, Left, Up order)	Queue after expansion (abbreviated)
1	S (0,0)	(0,1), (1,0)	(0,1), (1,0)
2	(0,1)	(0,2)	(1,0), (0,2)
3	(1,0)	(2,0)	(0,2), (2,0)
4	(0,2)	(0,3)	(2,0), (0,3)
5	(2,0)	(2,1), (3,0)	(0,3), (2,1), (3,0)
6	(0,3)	(0,4), (1,3)	(2,1), (3,0), (0,4), (1,3)
7	(2,1)	(2,2), (3,1)	(3,0), (0,4), (1,3), (2,2), (3,1)
8	(3,0)	(4,0)	(0,4), (1,3), (2,2), (3,1), (4,0)
9	(0,4)	(1,4)	(1,3), (2,2), (3,1), (4,0), (1,4)
10	(1,3)	(2,3)	(2,2), (3,1), (4,0), (1,4), (2,3)
11	(2,2)	(blocked neighbours skipped; (2,3) already discovered)	(3,1), (4,0), (1,4), (2,3)
12	(3,1)	(4,1)	(4,0), (1,4), (2,3), (4,1)
13	(4,0)	(4,1) already discovered	(1,4), (2,3), (4,1)
14	(1,4)	(2,4)	(2,3), (4,1), (2,4)
15	(2,3)	(3,3) blocked; (2,4) already discovered	(4,1), (2,4)
16	(4,1)	(4,2)	(2,4), (4,2)
17	(2,4)	(3,4)	(4,2), (3,4)
18	(4,2)	(4,3)	(3,4), (4,3)
19	(3,4)	(4,4) = G	(4,3), G
20	(4,3)	G already discovered	G

One shortest path: (0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

Path cost = 8 moves × 1 per move = 8. This result is for the illustrative obstacle arrangement only. If the instructor intended a different grid, use that layout and repeat the BFS trace.

3. Autonomous Rescue Robot in a Partially Observable Building

The rescue robot must move from S to survivor location G in a grid with blocked cells and limited sensing. It can observe only adjacent cells, movement costs 1, entering a risky zone adds 2, and the robot must update its knowledge online while prioritizing safe navigation.

a) How the constraints affect decisions

Constraint	Effect on decision-making
Four-direction movement	Only up, down, left and right successors may be considered; diagonal shortcuts are not allowed.
Blocked cells	The robot must reject known blocked cells and avoid routes that pass through them.
Limited sensing	The complete map is not initially available; the robot must infer the local state and explore carefully.
Movement cost 1	Each ordinary move adds 1 to accumulated path cost.
Risk-zone surcharge +2	Entering a risky cell costs 1 + 2 = 3, so the robot should compare routes by total cost, not distance alone.
Dynamic knowledge	New observations may reveal an obstacle or a safe passage, requiring the internal map and plan to be updated.
Safety requirement	The robot should favour verified traversable routes and avoid unnecessary exposure to risk.

Cost model

For a path P, total cost can be expressed as: Cost(P) = number of moves + 2 × number of risky cells entered. This formula assumes the extra risk cost is charged once whenever a risky cell is entered; if the problem defines risk differently, the cost model should be adjusted.

b) Suitable approach

Use an online search agent with a model-based internal map, and apply Uniform Cost Search (UCS) to the currently known traversable graph. UCS expands the frontier node with the smallest accumulated cost g(n). When the robot discovers new information, it updates the map and replans. This is more appropriate than plain BFS when risky cells have additional costs, because not all transitions have equal cost.

3. Rescue Robot — Search Procedure and AI Concepts

Online UCS procedure

```
INITIALISE map with current cell S and unknown neighbouring cells
WHILE survivor G has not been reached:
  observe adjacent cells
  update internal map: free, blocked, risky, or unknown
  construct graph from currently known safe/traversable cells
  run UCS from current position toward G using accumulated cost g(n)
  IF a route is known:
     execute the first safe move on the least-cost route
  ELSE:
     choose a safe frontier cell that may reveal new space
  update position and path cost
  IF new obstacle or risk information appears:
     update map and replan
STOP when G is reached; report total cost and route
```

Decision cycle

Stage	What the robot does	AI concept
Perceive	Sense adjacent cells and identify blocked, open or risky spaces	Perception
Update	Revise internal map and current position estimate	Model-based agent
Plan	Compare known paths using accumulated cost	Search / rational choice
Act	Take the next safe step on the selected route	Action
Learn and replan	Use new observations to revise route decisions	Online search

c) Intelligent agents, rationality and environment type

Concept	Application to rescue robot
Intelligent agent	Receives percepts through sensors and acts through movement controls to reach the survivor.
Rationality	Chooses the action expected to achieve the goal with the lowest known cost while respecting safety and available information.
Partially observable	The robot sees adjacent cells only; it cannot initially know the whole building layout.
Sequential	Each movement affects later options and the final route cost.
Stochastic / uncertain	Conditions may be unknown and observations can reveal unexpected obstacles or hazards.
Dynamic knowledge / online	The internal map changes as the robot explores, so planning is repeated when needed.

3. Rescue Robot — Justification and Worked Cost Example

d) Why online search with UCS is suitable

- Partial observability: the robot can plan only from currently available information, then update its map after sensing adjacent cells.
- Different route costs: a normal move costs 1, while entering a risky zone costs 3 in total. UCS handles non-negative, non-uniform costs directly.
- Safety-aware choices: risky cells can be represented with larger transition costs, discouraging them when a safer, lower-cost route exists.
- Replanning: when a blocked cell or risk zone is discovered, the robot updates the graph and searches again.
- Optimality on the known graph: with non-negative edge costs, UCS finds a least-cost path to the goal in the graph it currently knows. It cannot guarantee a globally optimal real-world route if important parts of the map remain unknown.

Illustrative route-cost comparison

The source question does not provide a specific rescue grid or mark which cells are risky. The table below is a generic example to demonstrate the cost rule, not a claimed solution for an unspecified map.

Candidate route	Moves	Risky cells entered	Total cost
Route A: shorter but crosses risk	5	2	5 + (2 × 2) = 9
Route B: longer, avoids risk	8	0	8 + (0 × 2) = 8
Route C: longer with one risky cell	7	1	7 + (1 × 2) = 9

In this example, Route B is preferred because its total cost is 8, lower than the other candidate routes, even though it uses more moves than Route A. This demonstrates why distance alone is not enough when risk has an additional penalty.

Implementation considerations

- Maintain a map with cell status: unknown, free, blocked or risky.
- Do not treat unknown cells as confirmed safe; sense before entering when possible.
- Track accumulated path cost separately from Manhattan distance.
- Replan after new obstacle information or a change in risk assessment.
- Stop and request assistance if no safe reachable frontier remains.

Summary and Assessment Checklist

Summary of solutions

Question	Main method	Key result
1. Doctor shifts	Constraint satisfaction + backtracking	D2—Morning, D1—Afternoon, D3—Night.
2. 5×5 robot grid	Breadth-First Search	Illustrative grid yields an 8-move shortest path; exact answer depends on obstacle layout.
3. Rescue robot	Online search agent + UCS	Minimizes accumulated movement and risk costs on the known map, replanning as information changes.

Submission checklist

- Show the constraint checks and backtracking decisions for the doctor schedule.
- Confirm the intended 5×5 grid and obstacle positions before using the BFS path and cost.
- Explain that Manhattan distance is a heuristic but does not control the FIFO ordering of BFS.
- Use the stated risk surcharge consistently in rescue-route cost calculations.
- State assumptions clearly whenever the question paper omits a grid or risk layout.

Rubric supplied with the question paper

Criterion	Weightage	What a strong response demonstrates
Problem understanding	25%	Defines the problem and identifies all relevant constraints accurately.
Constraint analysis	25%	Analyses constraints effectively and prioritises them logically.
Solution development	25%	Develops feasible solutions satisfying all constraints.
Application of concepts	15%	Applies appropriate concepts, methods or tools.
Justification	10%	Provides strong justification with logical reasoning.